Generative AI Intro: LLMs (20 points)

Objective: Delve into an introduction to the subcategory of Generative AI called Large Language Models (LLMs) and reflect on a real-world scenario.

All answers must be written using complete sentences. When you are finished, download as a .pdf and upload to Canvas under the Week 10 Submission.

Part I: Watch one of the following videos. If you do not have headphones, please turn off your sound and turn on the subtitles.

Google Cloud Tech: Introduction to Large Language Models (~15 minutes)
An introduction to large language models, including use cases, Prompt Tuning, and GenAI development tools.

Google Cloud Tech: Introduction to Generative AI (~22 minutes)
A definition of Generative AI and how it works, along with common applications for Generative AI.

Google for Developers: How Are Large Language Models Trained? (~10 minutes)
An introduction to the two core phases of how large language models are trained, pre-training and post-training.

Written Response:

1. From what you have learned, compare/contrast coding in Java (code executes exact, conditional logic) with probabilistic AI models (systems predict tokens based on statistical weights).

   Java follows exact instructions and conditional logic to produce predictable results, while AI models use statistics and probabilities to predict the most likely next tokens and generate appropriate responses.

2. When a large language model is used for coding, name one challenge that could affect testing and debugging the software.

   One thing that can affect AI Code generation is code hallucination/hallucination. Hallucination is when an AI presents false or misleading information in a completely confident way. This also goes in hand with generating code with AI. Using this “hallucinated” code can cause major bugs and may break things when working with a professional codebase.

Part II: Select **ONE** of the following tech scenarios to analyze:

Scenario A: Algorithmic Bias — An AI resume screener rates applicants from certain zip codes lower because of historical hiring data.

Scenario B: Generative Code Licensing — An AI coding assistant outputs verbatim blocks of open-source software code without required attribution.

Scenario C: Prompt Injection Attacks — A user manipulates a customer-service chatbot into executing unauthorized commands or revealing system instructions.

Written Response: Answer based on the scenario that you picked.

1. What is the root cause of the issue from a computer science perspective (training data, token probability, or input validation)?

   The root cause of prompt injection attacks from a computer science perspective is a lack of input validation and the inability of the LLM to separate user instructions from system data.

2. As a software engineer, what step or guardrail would you build into the system development lifecycle to prevent the scenario from happening?

   I would implement a strict input validation layer that scans for prompts that may be intended to bypass security restrictions or other malicious acts, then shut them down so no further harm can be caused. Somewhat unrelated to LLMs, but you could also implement a consequence system that can potentially ban the user from the AI services if they mean to cause harm. This could incentivize them to not do this specific action.